

Fusión Sensorial LiDAR–Cámara para la Navegación Autónoma del Yahboom MicroROS Robot Car ROS2 en el Gran Prix CappyTown

El Qhapaq Ñan: El Laberinto del Chaski

AGUILAR CONTRERAS Angel Jesus

CABALLERO SALAZAR Mattias Lincoln

NECIOSUP SAAVEDRA Leslie Jazmin

TICONA SANCHEZ Camila Danna

Grupo 1 – Sección 002

Facultad de Ingeniería, Universidad ESAN

Lima, Perú

Resumen—Este trabajo presenta el diseño e implementación de un sistema de navegación móvil autónoma para el reto final Gran Prix CappyTown. La solución integra el LiDAR MS200, la cámara frontal, la odometría y una máquina de estados finitos ejecutada en ROS 2 Humble sobre el robot Yahboom MicroROS Robot Car. La navegación geométrica se basa en seguimiento de pared derecha, ajuste de una recta sobre los puntos laterales del LiDAR, detección de intersecciones y recuperación ante callejones sin salida. La percepción semántica emplea segmentación del color rojo en HSV, filtrado morfológico y validación temporal para reconocer señales de PARE y forzar una detención completa de tres segundos. El arbitraje asigna a la cámara prioridad para detener y al LiDAR prioridad para el movimiento y la evitación de colisiones. Se propone un protocolo de 10 corridas para comparar exploración y *time attack* mediante tiempo, longitud de ruta, eficiencia, colisiones y cumplimiento de señales. La arquitectura modular facilita la calibración independiente de percepción, control, navegación y registro de métricas.

Index Terms—ROS 2, LiDAR, visión por computador, navegación autónoma, seguimiento de pared, fusión sensorial, máquina de estados, Yahboom MicroROS Robot Car.

I. INTRODUCCIÓN

El Gran Prix CappyTown plantea la navegación completamente autónoma de un robot móvil desde la celda de inicio A4 hasta la meta F1, dentro de un laberinto de 360 cm por 240 cm, organizado en una rejilla de 6×4 celdas de 60 cm. La pista contiene rutas alternativas, intersecciones, callejones sin salida y señales de PARE que cambian de posición entre corridas. Por ello, el robot debe combinar información geométrica del LiDAR con información semántica proveniente de la cámara [1].

El reto no consiste únicamente en alcanzar la meta. También exige evitar contactos con las paredes, respetar cada señal de PARE, reducir el tiempo de recorrido y aproximarse a la ruta más corta. Estas condiciones requieren una arquitectura capaz de percibir, decidir y actuar en tiempo real, sin que varios nodos envíen órdenes contradictorias al sistema de locomoción.

El Grupo 1 de la sección 002 desarrolló una solución modular en ROS 2 Humble para el Yahboom MicroROS Robot Car. El

sistema separa el procesamiento LiDAR, el control de pared, la detección visual, la lógica de estados y el registro de métricas. Como regla de seguridad, el nodo de navegación es el único publicador de comandos finales en `/cmd_vel`; los demás nodos entregan mediciones o sugerencias de movimiento.

II. OBJETIVOS

II-A. Objetivo general

Diseñar e implementar un sistema de navegación autónoma con fusión de cámara, LiDAR y odometría que permita al MicroROS Robot Car recorrer el laberinto Gran Prix CappyTown desde A4 hasta F1, evitando colisiones y respetando las señales de PARE.

II-B. Objetivos específicos

- Procesar el escaneo de 360° para obtener distancias frontales y laterales útiles para navegación.
- Mantener una separación segura y un desplazamiento paralelo respecto de la pared derecha.
- Detectar intersecciones, caminos libres y callejones sin salida.
- Detectar señales rojas de PARE mediante visión por computador y ejecutar una detención de aproximadamente 3 s.
- Centralizar las decisiones de movimiento mediante una máquina de estados finitos.
- Registrar tiempo, distancia recorrida, eficiencia, eventos de PARE, callejones visitados y eventos de riesgo de colisión.

III. ESCENARIO Y REQUERIMIENTOS DEL RETO

La Tabla I resume las condiciones técnicas consideradas en el diseño. La modalidad oficial integra percepción LiDAR, percepción por cámara, control, odometría, visualización y métricas [1].

Cuadro I
CARACTERÍSTICAS PRINCIPALES DEL ESCENARIO

Elemento	Descripción
Pista	360 cm × 240 cm; rejilla 6 × 4
Celda	60 cm × 60 cm
Inicio y meta	A4 y F1, respectivamente
Robot	Yahboom MicroROS Robot Car, chasis Ackermann
Computador	Raspberry Pi 5
Middleware	ROS 2 Humble
Sensores	LiDAR MS200, cámara frontal y odometría
Ronda 1	Exploración sin conocimiento previo
Ronda 2	Time attack con parámetros optimizados
Regla visual	Detención completa de aproximadamente 3 s ante PARE
Seguridad	Cero contactos con paredes u obstáculos

IV. ARQUITECTURA DEL SISTEMA

La arquitectura sigue el principio de responsabilidad única. Cada nodo procesa una función específica y se comunica mediante tópicos ROS 2. El archivo de lanzamiento inicia el procesador LiDAR, el seguidor de pared, la máquina de estados, el detector de PARE y el registrador de métricas [?].

La Tabla II presenta las responsabilidades principales.

Cuadro II
RESPONSABILIDAD DE LOS NODOS PRINCIPALES

Nodo	Función
lidar_processor	Convierte /scan en distancias por zona y ajusta la pared derecha mediante regresión.
wall_follower	Calcula una sugerencia de velocidad lineal y angular para mantener paralelismo y distancia lateral.
stop_sign_detector	Segmenta rojo en HSV, valida contornos y publica la detección confirmada.
state_machine	Fusiona sensores, decide la maniobra y es el único nodo que publica en /cmd_vel.
metrics_logger	Acumula distancia y eventos; escribe una fila por corrida en formato CSV.

V. METODOLOGÍA

V-A. Procesamiento del LiDAR

El LiDAR entrega pares (r_i, θ_i) en el marco del sensor. Antes de usar las mediciones se corrige el desfase angular de montaje y, si corresponde, la inversión entre izquierda y derecha. Los puntos seleccionados del costado derecho se transforman al marco del robot mediante

$$x_i = r_i \cos(\theta_i), \quad y_i = r_i \sin(\theta_i). \quad (1)$$

Sobre estos puntos se ajusta la recta

$$y = mx + b \quad (2)$$

mediante mínimos cuadrados con rechazo iterativo de valores atípicos. El ángulo de la pared y la distancia perpendicular al origen se calculan como

$$\alpha = \arctan(m), \quad d = \frac{|b|}{\sqrt{m^2 + 1}}. \quad (3)$$

El uso de múltiples puntos reduce la influencia del ruido frente a una estimación basada únicamente en dos rayos [3].

Además, el escaneo se divide en ventanas para obtener las distancias *front*, *right-front*, *right*, *right-rear* y *left*. Estas variables permiten evaluar caminos libres, proximidad frontal y alineación después de cada giro.

V-B. Seguimiento de pared derecha

El controlador busca mantener una distancia objetivo d^* respecto de la pared. El error lateral se define como

$$e_d = d^* - d. \quad (4)$$

La orden angular sugerida se obtiene combinando corrección de orientación y corrección de distancia:

$$\omega = \text{sat}(K_\alpha \alpha + K_d e_d, -\omega_{\text{máx}}, \omega_{\text{máx}}). \quad (5)$$

A diferencia de un control que alterna entre ángulo y distancia, la suma de ambos términos permite corregir simultáneamente el paralelismo y la separación. Cuando no existe una pared derecha confiable, se mantiene el rumbo con un control proporcional sobre el *yaw* de odometría. Si un obstáculo se encuentra demasiado cerca al frente, la velocidad lineal se anula [4].

V-C. Detección de intersecciones y callejones

Una dirección se considera libre cuando la distancia de su sector supera el umbral correspondiente. Para evitar decisiones por una lectura aislada, la máquina de estados evalúa cinco muestras y exige al menos cuatro coincidencias. La prioridad de decisión implementada es:

$$\text{derecha} \rightarrow \text{frente} \rightarrow \text{izquierda} \rightarrow \text{giro de } 180^\circ. \quad (6)$$

La última opción identifica un callejón sin salida. Esta regla de mano derecha es reactiva, no requiere un mapa previo y garantiza la salida de un laberinto simplemente conexo, siempre que el seguimiento de pared se mantenga de forma continua.

Algorithm 1 Decisión en una intersección

- 1: Detener el robot y confirmar sectores libres
- 2: **if** derecha libre **then**
- 3: girar a la derecha
- 4: **else if** frente libre **then**
- 5: continuar recto
- 6: **else if** izquierda libre **then**
- 7: girar a la izquierda
- 8: **else**
- 9: registrar callejón y girar 180°
- 10: **end if**
- 11: alinear con la nueva pared de referencia

V-D. Giros y odometría en chasis Ackermann

El MicroROS Robot Car no puede girar sobre su propio eje. Por esta razón, los giros se realizan como arcos de baja velocidad lineal y velocidad angular controlada. El radio aproximado es

$$R \approx \frac{v}{|\omega|}. \quad (7)$$

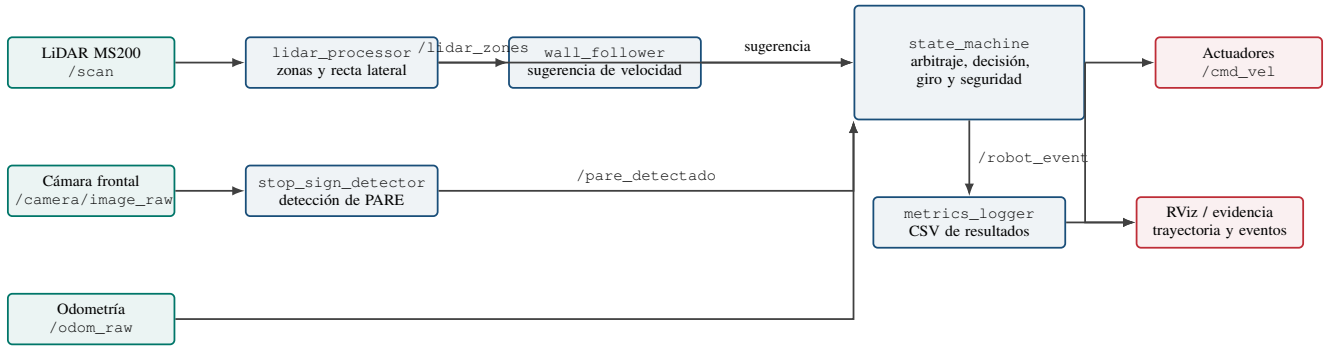


Figura 1. Arquitectura funcional de nodos y tópicos del sistema.

El giro termina cuando el error entre el yaw objetivo y el medido es menor que la tolerancia configurada. Después se ejecuta una fase de alineación comparando las distancias delantera y trasera del costado derecho.

Para reducir el error sistemático del odómetro se aplican factores de escala a la distancia y al ángulo:

$$d_c = k_d d_{odom}, \quad \psi_c = k_\psi \psi_{odom}. \quad (8)$$

Los valores iniciales utilizados fueron $k_d = 0,9474$ y $k_\psi = 0,9899$, sujetos a recalibración en la pista real [6].

V-E. Detección visual de señales de PARE

La imagen BGR se convierte al espacio HSV. Debido a que el rojo aparece en los dos extremos del eje de matiz, se emplean dos intervalos:

$$H_1 \in [0, 10], \quad S \in [120, 255], \quad V \in [70, 255], \quad (9)$$

$$H_2 \in [170, 180], \quad S \in [120, 255], \quad V \in [70, 255]. \quad (10)$$

Las máscaras se combinan y se someten a apertura y cierre morfológico con un kernel de 5×5 . Posteriormente, los contornos se filtran por área y relación de aspecto. Una señal se confirma después de tres cuadros positivos consecutivos y se elimina después de cinco cuadros sin detección. Este filtrado temporal disminuye falsos positivos provocados por reflejos u objetos rojos aislados [5].

V-F. Fusión sensorial y arbitraje

La cámara y el LiDAR cumplen funciones distintas. El LiDAR determina si el robot puede avanzar, girar o debe evitar una colisión; la cámara determina si corresponde detenerse por una regla de tránsito. Si ambos sensores entregan órdenes incompatibles, la prioridad es:

$$\text{PARE o riesgo frontal} > \text{orden de avance}. \quad (11)$$

Así, una detección confirmada de PARE fuerza velocidad nula durante 3 s, incluso cuando el pasillo está libre.

V-G. Máquina de estados

La Fig. 2 resume la lógica principal. La pausa previa al giro separa visualmente el final del avance y el inicio de la maniobra. El estado DETENIDO actúa como protección frente a un número excesivo de celdas recorridas.

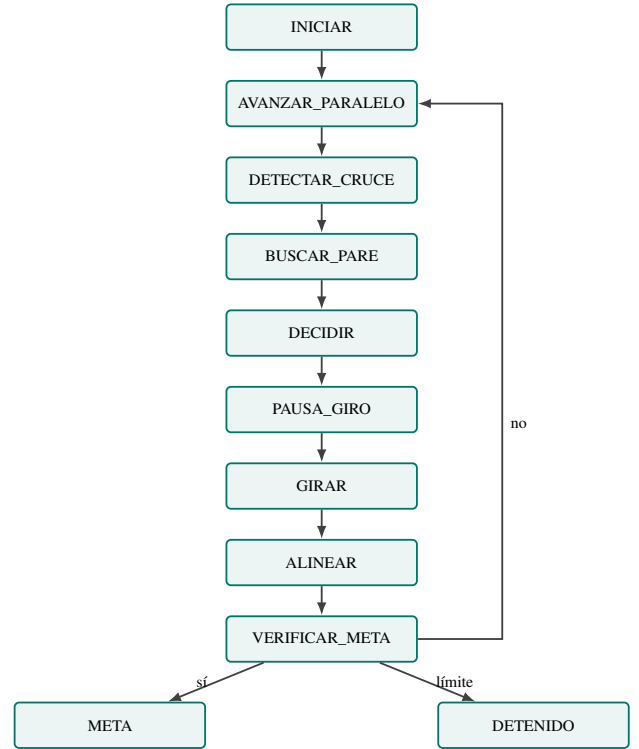


Figura 2. Máquina de estados del sistema de navegación.

V-H. Rondas de competencia

En la Ronda 1 se utiliza la estrategia reactiva de seguimiento de pared derecha y se registran todas las decisiones y eventos. En la Ronda 2 se conservan los mismos mecanismos de seguridad, pero pueden ajustarse velocidades y tolerancias con base en la calibración previa. Si el trazado se mantiene sin cambios, un módulo adicional de grafo y búsqueda en anchura puede aprovechar el recorrido previo; si las paredes

se reconfiguran, el mapa debe invalidarse y se mantiene la navegación reactiva para no confiar en información obsoleta.

VI. IMPLEMENTACIÓN Y PARÁMETROS

La solución se organizó en dos paquetes: `capytown_interfaces`, para los mensajes personalizados, y `capytown_granprix`, para los nodos de navegación. Los mensajes `LidarZones` y `RobotEvent` desacoplan la percepción de la lógica de control y del registro de resultados.

Los parámetros principales empleados como punto de partida se muestran en la Tabla III. Deben recalibrarse en función de iluminación, fricción, montaje del LiDAR y respuesta real de la dirección.

Cuadro III
PARÁMETROS INICIALES DE OPERACIÓN

Parámetro	Valor
Distancia objetivo a pared	0,12 m
Velocidad lineal base	0,15 m s ⁻¹
Ganancia angular K_α	2,0
Ganancia de distancia K_d	2,0
Velocidad angular máxima	0,60 rad s ⁻¹
Frente mínimo del seguidor	0,15 m
Umbral de pared frontal	0,25 m
Umbral de frente libre	0,35 m
Umbral de lado libre	0,40 m
Umbral preventivo de colisión	0,10 m
Velocidad lineal de giro	0,08 m s ⁻¹
Velocidad angular de giro	0,50 rad s ⁻¹
Tolerancia de giro	4°
Tiempo de PARE	3 s

VII. PROCEDIMIENTO EXPERIMENTAL

La validación se dividió en pruebas unitarias y corridas completas:

1. Calibración del desfase y sentido del LiDAR mediante objetos ubicados a distancias conocidas en cada costado.
2. Calibración de odometría mediante recorridos rectos y giros de 90° medidos físicamente.
3. Prueba de seguimiento de pared en un pasillo recto de 60 cm de ancho.
4. Prueba de giro derecho, giro izquierdo y retorno de 180°.
5. Prueba de detección de PARE bajo diferentes distancias e iluminación.
6. Corridas completas desde A4 hasta F1, registrando tópicos, video, trayectoria de RViz y archivo CSV.

Se propone ejecutar 10 corridas: cinco de exploración y cinco de *time attack*. Antes de cada corrida se registra el número real de señales, la configuración de paredes y cualquier obstáculo adicional.



Figura 3. Robot Yahboom MicroROS Robot Car durante una prueba en el laberinto.

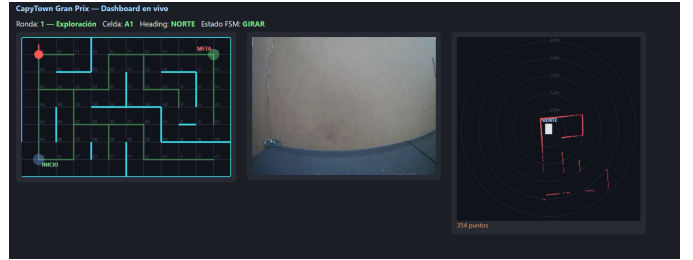


Figura 4. Visualización de la trayectoria y percepción del sistema en RViz.

VIII. MÉTRICAS DE EVALUACIÓN

La eficiencia de ruta se calcula como

$$\eta = \frac{L_{\text{ptima}}}{L_{\text{recorrida}}}, \quad 0 < \eta \leq 1. \quad (12)$$

Un valor cercano a uno indica que el recorrido se aproxima a la ruta de referencia. Para la detección de PARE se consideran verdaderos positivos (VP), falsos positivos (FP) y falsos negativos (FN):

$$\text{Precisin} = \frac{VP}{VP + FP}, \quad (13)$$

$$\text{Recall} = \frac{VP}{VP + FN}, \quad (14)$$

$$F_1 = 2 \frac{\text{Precisin} \cdot \text{Recall}}{\text{Precisin} + \text{Recall}}. \quad (15)$$

El registrador acumula la distancia a partir de odometría y cuenta los eventos publicados por la máquina de estados. El campo de falsos PARE debe verificarse manualmente contra el video, porque el robot no posee una referencia externa que indique si un objeto rojo corresponde realmente a una señal [7].

IX. RESULTADOS Y DISCUSIÓN

De las 10 pruebas realizadas, el robot alcanzó la meta en XX corridas. El tiempo promedio fue de XX s en la Ronda 1 y de XX s en la Ronda 2. La eficiencia promedio cambió de XX a XX, lo que permite determinar si la calibración y el ajuste de velocidades mejoraron realmente el desempeño.

En percepción visual se obtuvieron una precisión de XX, un *recall* de XX y un valor F_1 de XX. El número de PARE respetados debe coincidir con las detecciones verdaderas para confirmar que la máquina de estados ejecutó la detención completa. Los falsos positivos se analizan revisando el video y comparándolo con los eventos PARE_DETECTADO.

La principal ventaja del ajuste de recta lateral fue la estabilidad del seguimiento, porque el control no dependió de un único rayo susceptible a ruido. La confirmación por consenso también redujo giros erróneos en intersecciones. Sin embargo, la precisión global continuó dependiendo de la odometría, de la adherencia de las ruedas y de la disponibilidad de espacio para ejecutar arcos con un chasis Ackermann.

Debe diferenciarse entre una colisión física y un evento preventivo. En la implementación, una distancia frontal inferior al umbral de seguridad genera un evento y detiene el robot antes del contacto. Para la evaluación oficial, los contactos reales deben confirmarse mediante observación o video, evitando contabilizar automáticamente toda detención preventiva como choque.

X. LIMITACIONES

- La posición de celda se estima por odometría y conteo de movimientos; un deslizamiento acumulado puede desincronizar la celda lógica de la real.
- El chasis Ackermann requiere espacio para los giros y no puede rotar en el sitio.
- La segmentación HSV es sensible a cambios de iluminación, saturación de la cámara y objetos rojos similares a la señal.
- El número de falsos PARE requiere validación manual con el video.
- Un mapa aprendido en la primera ronda no debe reutilizarse si la pista fue reconfigurada.

XI. CONCLUSIONES

Se desarrolló una arquitectura modular de navegación autónoma que fusiona LiDAR, cámara y odometría en ROS 2. El procesador LiDAR obtuvo una representación compacta del entorno, mientras que el ajuste de recta lateral permitió controlar simultáneamente la orientación y la distancia respecto de la pared derecha. La máquina de estados centralizó las decisiones y evitó comandos de movimiento contradictorios.

La detección de PARE mediante HSV, filtros morfológicos y confirmación temporal permitió integrar una regla semántica dentro de la navegación geométrica. El arbitraje priorizó las detenciones de seguridad y el cumplimiento de la señal frente a cualquier orden de avance.

La comparación entre rondas debe sustentarse con las métricas de la Tabla ???. Una mejora válida no depende solo

de reducir el tiempo, sino también de mantener cero colisiones, respetar todas las señales y aumentar la eficiencia de ruta. Como trabajo futuro se propone incorporar localización más robusta, actualización de mapas de ocupación, planificación A* o BFS cuando el trazado sea estable y detección visual basada en aprendizaje profundo para reducir la sensibilidad a la iluminación.

CONTRIBUCIÓN DE LOS INTEGRANTES

Integrante	Contribución principal
AGUILAR CONTRERAS Angel Jesus	Integración LiDAR, calibración y pruebas de percepción geométrica.
CABALLERO SALAZAR Matias Lincoln	Control de seguimiento de pared, giros y ajustes de movimiento.
NECIOSUP SAAVEDRA Leslie Jazmin	Detección visual de PARE y validación de cámara.
TICONA SANCHEZ Camila Danna	Integración ROS 2, FSM, RViz, métricas y documentación.

APÉNDICE A COMPILACIÓN Y EJECUCIÓN

Listing 1. Compilación de los paquetes ROS 2.

```
cd /root/yahboomcar_ws
colcon build --packages-select \
  capytown_interfaces capytown_granprix
source install/setup.bash
```

Listing 2. Ejecución de las dos rondas.

```
# Ronda 1: exploracion
ros2 launch capytown_granprix \
  granprix_bringup.launch.py ronda:=1

# Ronda 2: time attack
ros2 launch capytown_granprix \
  granprix_bringup.launch.py ronda:=2
```

Listing 3. Tópicos de verificación.

```
ros2 topic echo /lidar_zones
ros2 topic echo /robot_state
ros2 topic echo /robot_event
ros2 topic echo /pare_detectado
```

APÉNDICE B ARCHIVOS GRÁFICOS REQUERIDOS

Para compilar el documento con las evidencias, colocar en la misma carpeta del archivo .tex:

- university-color.png: logotipo de la universidad.
- carro_laberinto.jpg: fotografía del robot en la pista.
- rviz_granprix.png: captura de la interfaz de RViz.

Si una imagen no se encuentra, el documento mostrará automáticamente un recuadro con el nombre esperado del archivo.

REFERENCIAS

- [1] Universidad ESAN, “Gran Prix CapyTown: El Qhapaq Ñan, El Laberinto del Chaski,” documento del reto final de Robótica Móvil, Lima, Perú, 2026.
- [2] Open Robotics, “ROS 2 Humble Hawksbill Documentation,” documentación técnica de ROS 2, 2022.
- [3] Grupo 1, “Módulos `lidar_processor_node.py` y `lidar_utils.py`,” código fuente del proyecto `capytown_granprix`, Universidad ESAN, 2026.
- [4] Grupo 1, “Módulo `wall_follower_node.py`,” código fuente del proyecto `capytown_granprix`, Universidad ESAN, 2026.
- [5] Grupo 1, “Módulo `stop_sign_detector_node.py`,” código fuente del proyecto `capytown_granprix`, Universidad ESAN, 2026.
- [6] Grupo 1, “Módulo `state_machine_node.py`,” código fuente del proyecto `capytown_granprix`, Universidad ESAN, 2026.
- [7] Grupo 1, “Módulo `metrics_logger_node.py`,” código fuente del proyecto `capytown_granprix`, Universidad ESAN, 2026.
- [8] G. Bradski, “The OpenCV Library,” *Dr. Dobb's Journal of Software Tools*, 2000.